

PaCT: Patch-Based Multi-Channel Transformer for Predictive Process Monitoring

Mingyun Kang¹, Yongjae Lee¹, Gibeom Park² and Hyerim Bae^{1,*}

¹Major in Industrial Data Science & Engineering, Department of Industrial Engineering

²Department of Data Science, Graduate School of Data Science
Pusan National University

2, Busandaehak-ro 63beon-gil, Geumjeong-gu, Busan 46241, Republic of Korea

{alsrbs3345; yongzzai1102; qkrlqja9331}@pusan.ac.kr

*Corresponding author: hrbac@pusan.ac.kr

Received XXX 2026; accepted XXX 2026

ABSTRACT. *Predictive Process Monitoring (PPM) predicts the future behavior of ongoing cases from their observed prefixes. Existing deep learning approaches have advanced PPM through recurrent and Transformer-based architectures. However, many Transformer-based approaches process prefixes at the event level and fuse activity, timestamp, and attribute information before sequence modeling. This paper proposes PaCT, a patch-based multi-channel Transformer that groups aligned neighboring events into overlapping patches and applies separate Transformer encoders to the activity, timestamp, and attribute channels before activity-centered fusion. The fused patch-level context is passed to an autoregressive Transformer decoder. The decoder jointly performs next activity and suffix prediction, while a separate head estimates the remaining time under a unified multi-task objective. Experiments on nine real-life event logs show that PaCT achieves the best or near-best results on most logs across all three tasks.*

Keywords: Predictive Process Monitoring, Patches, Multi-Channel, Transformer.

1. Introduction. Business Process Management (BPM) [1] aims to analyze and improve organizational processes using data recorded by process-aware information systems (PAIS) during process execution [2]. A central challenge is not only understanding past behavior but also anticipating how ongoing cases will unfold. Predictive Process Monitoring (PPM) addresses this by using the partial execution history of a running case to predict its future behavior, such as next activities and the remaining time until completion [3]. Such predictions allow process stakeholders to intervene proactively, improving process efficiency and reducing the risk of undesirable outcomes.

Various methods have been developed for PPM. Early PPM studies relied on process mining [4], deriving predictions from process models and conformance-checking results [5]. However, their dependence on explicit process models limited their applicability to noisy or unstructured logs. The adoption of machine learning (ML) and deep learning (DL) shifted the focus to learning directly from event logs. While ML-based approaches [6] achieved strong predictive performance, recent studies have increasingly turned to DL-based methods to model sequential behavior directly from running cases. Among DL methods, recurrent architectures such as Long Short-Term Memory (LSTM) [7] and Gated Recurrent Units (GRUs) [8] gained traction for modeling the sequential structure of cases. Nevertheless, recurrent computation struggles to capture long-range dependencies efficiently. The Transformer [9] addresses this limitation through self-attention, relating distant events directly. Recent Transformer-based studies have also addressed multi-task learning and remaining time prediction [10, 11]. Despite these advances, such models still represent each prefix at the event level and fuse the activity, timestamp, and attribute information of each event into a single representation before encoding.

This event-level early-fusion design has two limitations. First, event-level encoding requires the

model to discover local multi-event patterns through successive attention layers. Patch-based encoding instead treats neighboring events as explicit representation units, allowing local subsequences to be modeled directly. Second, early fusion requires a shared encoder to model heterogeneous categorical and temporal information from the outset. PaCT preserves channel-specific transformations until patch-level fusion, allowing each information type to be encoded separately before integration.

To address these limitations, we propose PaCT, a patch-based multi-channel Transformer framework for predictive process monitoring. Rather than representing a running case as a flat sequence of events, PaCT constructs overlapping patches to explicitly model local event subsequences and independently encodes activity, timestamp, and attribute information before channel fusion. Trained with a unified multi-task objective, PaCT autoregressively generates the remaining activity suffix and estimates the remaining time. Experiments are conducted on nine real-life benchmark event logs following the evaluation protocol of [12] to investigate the effects of patch representation, channel encoding strategy, and patch size on predictive performance.

The remainder of this paper is organized as follows. Section 2 introduces the notation used to define event logs and prediction tasks. Section 3 presents the proposed method. Section 4 reports the experimental results. Finally, Section 5 concludes the paper.

2. Preliminaries. This section introduces the notation used throughout this paper to define event logs, traces, and the prediction tasks addressed in this study.

Definition 2.1 (Event). Let A be the set of activities, C be the set of cases, T be the time domain, and D_k be the value domain of an attribute indexed by $k \in \{2, \dots, K + 1\}$, where $K \geq 0$ is the number of attributes. An event is a tuple $e = (a, c, t, d^{(2)}, \dots, d^{(K+1)})$, where $a \in A$, $c \in C$, $t \in T$, and $d^{(k)} \in D_k$.

Definition 2.2 (Trace, Event Log). A trace is a finite non-empty sequence of events $\sigma = \langle e_1, \dots, e_n \rangle$ recording the execution of a single case, where all events share the same case component and are ordered by non-decreasing timestamp. An event log $L = \{\sigma_1, \dots, \sigma_N\}$ is a set of traces, where N is the number of cases and each event occurs at most once across the entire log.

Predictive Process Monitoring. Given a complete trace $\sigma = \langle e_1, \dots, e_n \rangle$, only a partial trace is available at prediction time. The observed prefix of length $\ell < n$ is denoted by $\sigma_{\leq \ell} = \langle e_1, \dots, e_\ell \rangle$, and the unobserved suffix by $\sigma_{> \ell} = \langle e_{\ell+1}, \dots, e_n \rangle$. This study considers three prediction tasks. Next activity prediction estimates the immediate next activity $a_{\ell+1}$. Suffix prediction estimates the future activity sequence after the prefix, including an end-of-sequence token EOS : $\mathbf{y} = \langle a_{\ell+1}, \dots, a_n, EOS \rangle$. Remaining time prediction estimates the time remaining until case completion, $r_\ell = t_n - t_\ell$. These three targets are defined from the same prefix-suffix split. The next activity is the first element of $\mathbf{y}$, whereas r_ℓ summarizes the time spanned by the unobserved suffix. This common structure allows all three tasks to share the same prefix representation.

3. Proposed Method. This section presents PaCT, and Figure 1 provides an overview of its architecture.

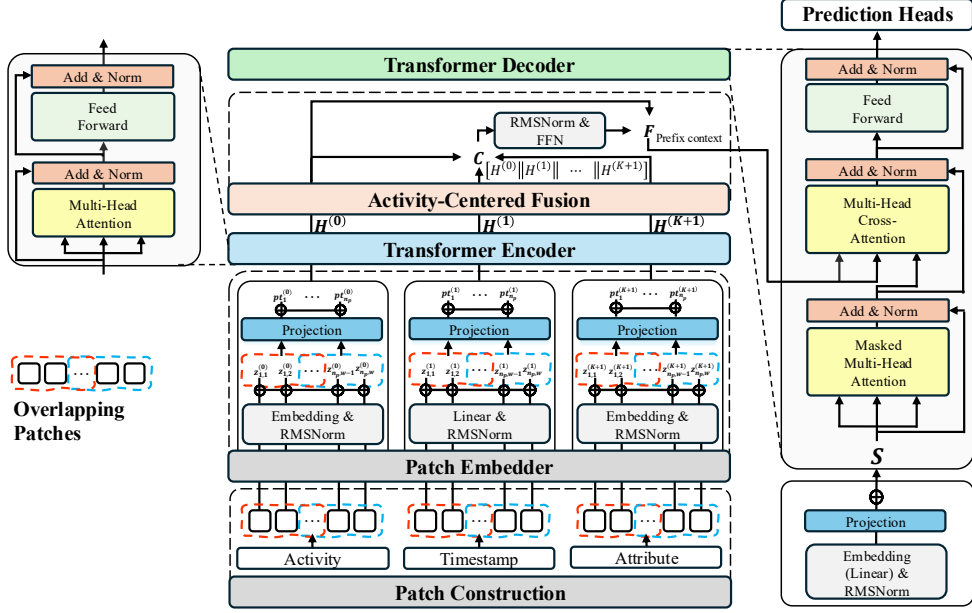


FIGURE 1. Proposed framework

3.1. Patch Construction. PaCT decomposes the prefix of length ℓ into an activity channel $\langle a_1, \dots, a_\ell \rangle$ ($k = 0$), a timestamp channel $\langle \tau_1, \dots, \tau_\ell \rangle$ ($k = 1$), and K event-attribute channels $\langle d_1^{(k)}, \dots, d_\ell^{(k)} \rangle$ for $k = 2, \dots, K + 1$. Each timestamp input $\tau = [TSS, TSP] \in \mathbb{R}^2$ is a z-score-normalized vector comprising the time since the case start (TSS) and the time since the previous event (TSP). A shared sliding window of size w and stride $st = \max(1, \lfloor w/2 \rfloor)$ is applied to all channels, producing n_p aligned patches per channel, with overlap when $w > 1$. Case-level attributes, which take a single value per case, are not decomposed into patches and are instead incorporated after channel fusion, as detailed in Section 3.4. Incomplete leading windows are left-padded, and a final tail-aligned patch is appended when necessary to cover the last observed event.

3.2. Patch Embedder. For activity and event-attribute channels ($k \in \{0, 2, \dots, K + 1\}$), categorical tokens are embedded using $E^{(k)}$, whereas the timestamp channel is linearly projected by W_τ :

$$z_{i,j}^{(k)} = \begin{cases} \text{RMSNorm}\left(E^{(k)}(x_{i,j}^{(k)})\right) + s_j, & k \in \{0, 2, \dots, K + 1\}, \\ \text{RMSNorm}\left(W_\tau x_{i,j}^{(k)}\right) + s_j, & k = 1, \end{cases} \quad (1)$$

where $x_{i,j}^{(k)}$ is the j -th token of the i -th patch and s_j is the within-patch positional embedding. The token representations are concatenated and projected to the model dimension:

$$pt_i^{(k)} = \mathbf{W}_{\text{proj}}^{(k)} \left[z_{i,1}^{(k)} \parallel \dots \parallel z_{i,w}^{(k)} \right] + p_i \in \mathbb{R}^{d_{\text{model}}}, \quad (2)$$

where $\mathbf{W}_{\text{proj}}^{(k)} \in \mathbb{R}^{d_{\text{model}} \times (w \cdot d_{\text{emb}})}$ is a bias-free linear projection, and p_i denotes the positional embedding of the i -th patch.

3.3. Transformer Encoder. Each channel is represented as a sequence of n_p patch-level vectors $pt^{(k)} = \langle pt_1^{(k)}, \dots, pt_{n_p}^{(k)} \rangle$ and processed by a dedicated Transformer encoder:

$$H^{(k)} = \text{Trn}_{\text{enc}}^{(k)}(pt^{(k)}) \in \mathbb{R}^{(n_p \times d_{\text{model}})}, \quad k = 0, \dots, K + 1. \quad (3)$$

Since the encoders do not share parameters across channels, each encoder can learn channel-specific patterns. Prefix self-attention is non-causal because all events in the observed prefix are available at prediction time. A patch-level padding mask prevents padded patches from contributing to the channel representations.

3.4. Activity-Centered Fusion. The encoded channel representations are concatenated along the feature axis as $\mathbf{C} = [H^{(0)} \| H^{(1)} \| \dots \| H^{(K+1)}]$ and passed through a SwiGLU feed-forward network (FFN) [13]. The activity representation $H^{(0)}$ is used as the residual backbone because activity transitions describe the structural control flow of a process, while timestamp and attribute channels provide complementary context.

$$F = H^{(0)} + \text{FFN}(\text{RMSNorm}(\mathbf{C})) \in \mathbb{R}^{(n_p \times d_{\text{model}})}. \quad (4)$$

When case-level attributes are available, their embeddings are concatenated and projected into a case-context vector, which is added to every fused patch representation before decoder cross-attention.

3.5. Transformer Decoder and Prediction Heads. The decoder applies causal self-attention over the embedded input sequence S and cross-attention to the fused prefix context F , producing a hidden state h_t at each step. During training, S embeds the last observed activity followed by the target suffix shifted right by one position; during inference, it is extended autoregressively until EOS is generated. An activity head whose weights are tied to $E^{(0)}$ maps each h_t to activity logits. The first generated activity is used for next activity prediction, the generated sequence is used for suffix prediction, and a separate linear head estimates the remaining time from h_1 .

3.6. Training Objective. PaCT jointly optimizes next activity prediction, suffix prediction, and remaining time prediction:

$$\mathcal{L} = \mathcal{L}_{\text{next}} + \mathcal{L}_{\text{suffix}} + \mathcal{L}_{\text{rem}}. \quad (5)$$

$\mathcal{L}_{\text{next}}$ is the cross-entropy at the first suffix position, while $\mathcal{L}_{\text{suffix}}$ averages cross-entropy from the second position through EOS , first within each sequence and then across sequences with valid targets beyond the first position. Padding is excluded. $\mathcal{L}_{\text{rem}}$ is the Huber loss ($\delta = 1$) [14] computed on remaining times normalized using statistics from the training fold. For datasets without timestamps, $\mathcal{L}_{\text{rem}}$ is omitted; all applicable loss terms are equally weighted.

4. Experiments. In this section, we first evaluate the predictive performance of PaCT on three PPM tasks. The main comparison evaluates both the single-channel and multi-channel variants against representative PPM baselines. We then analyze the effect of patch size on predictive performance and finally investigate the computational impact of patching in terms of runtime and peak allocated CUDA memory.

4.1. Experimental Setup. We evaluate PaCT on the nine real-life event logs included in the benchmark of Rama-Maneiro et al. [12]. Table 1 summarizes the characteristics of the datasets, including the numbers of cases, activities, events, variants, and available attributes. Attribute counts are reported as (case-level, event-level), excluding activity and timestamp. We denote the single-channel variant, which concatenates all prefix features before one encoder, by $PaCT_s$, and the proposed multi-channel variant by $PaCT_m$.

TABLE 1. Statistics of the nine event logs.

Dataset	#Cases	#Activities	#Events	Avg. Case Length	Avg. Case Duration	#Variants	#Attributes (Case, Event)
BPIC12 A	13,087	10	60,849	4.65	8.1	17	(1, 1)
BPIC12 O	5,015	7	31,244	6.23	17.2	168	(1, 1)
BPIC12 W Complete	9,658	6	72,413	7.50	11.4	2,263	(1, 1)
BPIC13 Closed	1,487	7	6,660	4.48	178.9	327	(0, 1)
BPIC13 Incidents	7,554	13	65,533	8.68	12.1	2,278	(3, 5)
Env Permit	1,434	27	8,577	5.98	5.4	116	(3, 2)
Helpdesk	4,580	14	21,348	4.66	40.9	226	(0, 1)
NASA	2,566	94	73,638	28.70	-	2,513	(0, 3)
Sepsis	1,049	16	15,214	14.49	28.5	845	(0, 1)

Following the benchmark protocol, all experiments are conducted using 5-fold cross-validation. Model selection is performed using validation loss under teacher forcing, whereas all reported test metrics are computed from autoregressively generated suffixes. Next activity prediction is evaluated using accuracy, suffix prediction using normalized Damerau-Levenshtein similarity, and remaining time prediction using MAE (days). The main results use a patch size of $w = 4$. PaCT uses $d_{\text{emb}} = 32$, $d_{\text{model}} = 64$, $n_{\text{head}} = 4$, two encoder layers per channel, two decoder layers, $d_{\text{ff}} = 128$, and $\text{dropout} = 0.1$. Models are trained for up to 30 epochs using AdamW ($\text{lr} = 3 \times 10^{-4}$, weight decay = 10^{-4}) with cosine annealing and a batch size of 128.

4.2. Main Results. Tables 2–4 compare PaCT with representative deep-learning-based PPM methods on next activity, suffix, and remaining time prediction. Across the three tasks, the PaCT variants achieve competitive results relative to the benchmark methods. $PaCT_m$ is generally stronger for next activity and suffix prediction, whereas the remaining time results are more similar between the two variants.

TABLE 2. Next activity prediction accuracy (%; higher is better).

	BPIC12 A	BPIC12 O	BPIC12 W Complete	BPIC13 Closed	BPIC13 Incidents	Env Permit	Helpdesk	NASA	Sepsis
Camargo et al. [15]	76.0	81.4	69.0	54.7	66.7	85.8	82.9	-	-
Evermann et al. [16]	75.8	79.4	67.5	58.8	66.8	76.2	83.7	20.4	40.0
Pasquadibisceglie et al. [17]	74.1	78.9	68.3	47.5	46.0	<u>86.7</u>	83.8	88.3	56.2
Tax et al. [18]	79.5	<u>82.3</u>	69.8	64.0	70.1	85.7	84.2	<u>89.4</u>	64.2
$PaCT_s$	<u>80.8</u>	<u>82.3</u>	<u>71.8</u>	<u>64.5</u>	71.3	81.6	85.4	86.7	63.5
$PaCT_m$	81.0	82.5	73.0	66.1	75.6	87.4	<u>85.2</u>	89.6	<u>64.1</u>

As shown in Table 2, the PaCT variants achieve the highest next activity accuracy on eight of the nine event logs, with $PaCT_m$ leading on seven. The two variants differ by at most 0.6 percentage points on four logs and are practically comparable, whereas $PaCT_m$ improves accuracy by 1.2 to 5.8 percentage points on the other five. These results show that multi-channel encoding provides meaningful gains on several logs while maintaining comparable performance on the others.

TABLE 3. Suffix prediction similarity (%; higher is better).

	BPIC12 A	BPIC12 O	BPIC12 W Complete	BPIC13 Closed	BPIC13 Incidents	Env Permit	Helpdesk	NASA	Sepsis
Camargo et al. [15]	65.4	68.5	<u>31.9</u>	66.4	52.9	<u>84.4</u>	91.1	-	-
Evermann et al. [16]	58.5	55.4	33.7	64.2	47.3	57.1	83.5	12.2	26.9
Pasquadibisceglie et al. [17]	28.7	45.9	11.3	52.8	36.1	39.2	46.2	8.4	7.4
Tax et al. [18]	46.0	49.7	7.9	58.2	33.4	81.6	87.0	23.2	11.6
$PaCT_s$	<u>70.7</u>	<u>70.0</u>	18.8	75.2	<u>57.6</u>	81.8	92.2	<u>48.5</u>	<u>28.5</u>
$PaCT_m$	71.9	70.3	17.6	<u>74.7</u>	65.9	87.5	<u>92.1</u>	49.0	34.5

Table 3 shows that a PaCT variant achieves the highest suffix similarity on eight of the nine event logs. The effect of multi-channel encoding is more variable for suffix prediction: $PaCT_m$ improves similarity by 5.7 to 8.3 percentage points on BPIC13 Incidents, Env Permit, and Sepsis, whereas the absolute differences on the other logs are at most 1.2 points. This variability may reflect error propagation in autoregressive suffix generation, where an early decoding error can influence subsequent predictions. Overall, $PaCT_m$ provides the stronger main configuration, while $PaCT_s$ remains competitive on logs where the difference between the variants is small.

TABLE 4. Remaining time prediction MAE (days; lower is better).

	BPIC12 A	BPIC12 O	BPIC12 W Complete	BPIC13 Closed	BPIC13 Incidents	Env Permit	Helpdesk	Sepsis
Camargo et al. [15]	9.0	15.2	10.4	257.1	28.1	8.3	18.5	-
Evermann et al. [16]	29.4	66.5	59.7	191.1	35.4	5.6	159.9	503.9
Pasquadibisceglie et al. [17]	<u>7.8</u>	<u>7.6</u>	<u>8.2</u>	159.2	12.4	4.1	8.2	34.0
Tax et al. [18]	20.3	68.2	127.4	172.8	30.1	4.1	68.9	421.6
$PaCT_s$	6.4	6.7	7.6	82.1	<u>10.2</u>	<u>3.9</u>	<u>5.2</u>	<u>33.0</u>
$PaCT_m$	6.4	6.7	7.6	<u>85.1</u>	8.6	3.8	5.1	30.6

Table 4 provides the most consistent evidence for the overall PaCT framework. A PaCT variant obtains the lowest MAE on all eight logs evaluated for remaining time. NASA is excluded because it does not contain timestamp information. The two variants produce identical results on the three BPIC12 logs; among the other five logs, $PaCT_m$ achieves a lower MAE on four, whereas $PaCT_s$ performs better on BPIC13 Closed. The between-variant differences are generally smaller than the improvements over the strongest baseline, suggesting that remaining time prediction depends more on the shared PaCT architecture than on the prefix-channel configuration.

4.3. Effect of Patch Size. Figure 2 and Table 5 examine NASA, Sepsis, and BPIC13 Incidents, the three logs with the longest average case lengths (28.70, 14.49, and 8.68 events), to assess patching.

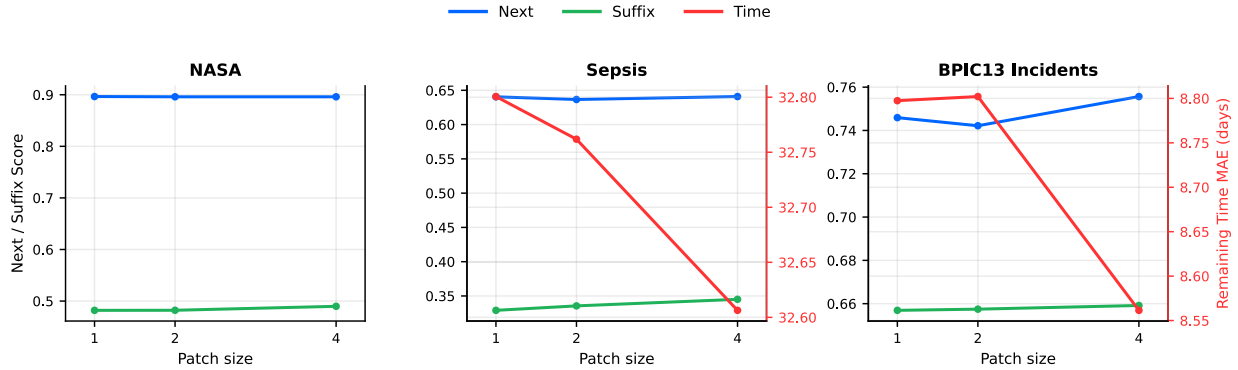


FIGURE 2. Patch-size sensitivity of multi-channel PaCT for $w \in \{1, 2, 4\}$ on three event logs.

TABLE 5. Computational effects of patching ($w = 4$ compared with $w = 1$).

Dataset	Training time per epoch (s), $w = 1 \rightarrow 4$	Inference time (s), $w = 1 \rightarrow 4$	Peak allocated CUDA memory (MB), $w = 1 \rightarrow 4$
NASA	20.49 $\rightarrow$ 18.73 (-8.6%)	31.49 $\rightarrow$ 26.85 (-14.7%)	502.1 $\rightarrow$ 308.2 (-38.6%)
Sepsis	2.92 $\rightarrow$ 2.98 (+2.3%)	5.17 $\rightarrow$ 5.23 (+1.2%)	351.3 $\rightarrow$ 228.8 (-34.9%)
BPIC13 Incidents	36.63 $\rightarrow$ 31.59 (-13.8%)	49.57 $\rightarrow$ 34.05 (-31.3%)	485.2 $\rightarrow$ 291.7 (-39.9%)

Figure 2 shows that, relative to $w = 1$, $w = 4$ changes next activity accuracy by less than one percentage point and maintains or slightly improves suffix similarity on all three logs. Remaining time MAE changes only modestly on Sepsis and BPIC13 Incidents, indicating that remaining time performance is largely preserved across patch sizes. Under the fixed 30-epoch schedule and identical batch-size conditions, Table 5 shows that $w = 4$ reduces peak allocated CUDA memory by 34.9% to 39.9% across all three logs. Training and inference times decrease on NASA and BPIC13 Incidents, whereas the small changes on Sepsis indicate comparable runtime. Overall, $w = 4$ preserves predictive performance and provides a consistent memory benefit, although its runtime benefit depends on the event log.

5. Conclusion. This paper presented PaCT, a patch-based multi-channel Transformer for predictive process monitoring. PaCT groups prefix events into overlapping patches and independently encodes activity, timestamp, and attribute channels before activity-centered fusion. Across nine benchmark event logs, a PaCT variant achieved the best result among the compared methods on eight logs for both next activity and suffix prediction and on all eight logs evaluated for remaining time. Multi-channel encoding provided clear benefits on several logs, although its relative effect remained task and event log dependent. On the three logs with the largest average case lengths, the default $w = 4$ configuration preserved predictive performance, consistently reduced peak allocated CUDA memory, and shortened runtime on two logs. Future work will investigate adaptive patch granularity based on observed prefix length and process characteristics, together with task-aware channel fusion that dynamically adjusts the contributions of activity, timestamp, and attribute channels.

Acknowledgment. This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No.RS-2023-00218913)

REFERENCES

- [1] M. Dumas, M. La Rosa, J. Mendling, and H. A. Reijers, *Fundamentals of Business Process Management*. Berlin Heidelberg: Springer, 2013. doi: 10.1007/978-3-642-33143-5.
- [2] W. Van Der Aalst et al., "Process Mining Manifesto," in *Business Process Management Workshops*, vol. 99, F. Daniel, K. Barkaoui, and S. Dustdar, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 169-194. doi: 10.1007/978-3-642-28108-2_19.
- [3] C. Di Francescomarino and C. Ghidini, "Predictive Process Monitoring," in *Process Mining Handbook*, vol. 448, W. M. P. Van Der Aalst and J. Carmona, Eds. Cham: Springer International Publishing, 2022, pp. 320-346. doi: 10.1007/978-3-031-08848-3_10.
- [4] A. Pika, W. M. P. Van Der Aalst, C. J. Fidge, A. H. M. Ter Hofstede, and M. T. Wynn, "Predicting Deadline Transgressions Using Event Logs," in *Business Process Management Workshops*, vol. 132, M. La Rosa and P. Soffer, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 211-216. doi: 10.1007/978-3-642-36285-9_22.
- [5] W. Van Der Aalst, *Process Mining*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2016.
- [6] B. Kang, D. Kim, and S. Kang, "Periodic performance prediction for real-time business process monitoring," *Ind. Manag. Data Syst.*, vol. 112, no. 1, pp. 4-23, Jan. 2012.
- [7] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735-1780, Nov. 1997. doi: 10.1162/neco.1997.9.8.1735.
- [8] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling," 2014, arXiv. doi: 10.48550/ARXIV.1412.3555.
- [9] A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, 2017.
- [10] J. Wang, J. Huang, X. Ma, Z. Li, Y. Wang, and D. Yu, "MTLFormer: Multi-Task Learning Guided Transformer Network for Business Process Prediction," *IEEE Access*, vol. 11, pp. 76722-76738, 2023. doi: 10.1109/ACCESS.2023.3298305.
- [11] A. N. Iman et al., "Predictive Process Monitoring for Remaining Time Prediction with Transfer Learning," *ICIC Express Lett.*, vol. 18, no. 8, pp. 851-858, Aug. 2024. doi: 10.24507/icicel.18.08.851.
- [12] E. Rama-Maneiro, J. C. Vidal, and M. Lama, "Deep Learning for Predictive Business Process Monitoring: Review and Benchmark," *IEEE Trans. Serv. Comput.*, vol. 16, no. 1, pp. 739-756, 2023. doi: 10.1109/TSC.2021.3139807.
- [13] N. Shazeer, "GLU Variants Improve Transformer," 2020, arXiv. doi: 10.48550/ARXIV.2002.05202.
- [14] P. J. Huber, "Robust Estimation of a Location Parameter," *Ann. Math. Stat.*, vol. 35, no. 1, pp. 73-101, Mar. 1964. doi: 10.1214/aoms/1177703732.
- [15] M. Camargo, M. Dumas, and O. González-Rojas, "Learning Accurate LSTM Models of Business Processes," in *Business Process Management*, vol. 11675, T. Hildebrandt, B. F. Van Dongen, M. Röglinger, and J. Mendling, Eds. Cham: Springer International Publishing, 2019, pp. 286-302. doi: 10.1007/978-3-030-26619-6_19.
- [16] J. Evermann, J.-R. Rehse, and P. Fettke, "Predicting process behaviour using deep learning," *Decis. Support Syst.*, vol. 100, pp. 129-140, Aug. 2017. doi: 10.1016/j.dss.2017.04.003.
- [17] V. Pasquadibisceglie, A. Appice, G. Castellano, and D. Malerba, "Using Convolutional Neural Networks for Predictive Process Analytics," in *2019 International Conference on Process Mining (ICPM)*, Aachen, Germany: IEEE, Jun. 2019, pp. 129-136. doi: 10.1109/ICPM.2019.00028.
- [18] N. Tax, I. Verenich, M. La Rosa, and M. Dumas, "Predictive Business Process Monitoring with LSTM Neural Networks," in *Advanced Information Systems Engineering*, vol. 10253, E. Dubois and K. Pohl, Eds. Cham: Springer International Publishing, 2017, pp. 477-492. doi: 10.1007/978-3-319-59536-8_30.